Using Asset Files

Our current tool setup uses the Propshaft gem to deliver general asset files to Rails. Propshaft will serve image, font, or other static asset files to your Rails code. For use of asset files in your JavaScript code, we can use esbuild.

Let's talk about Propshaft first.

Propshaft's goal in life is to allow you easy access to static asset files, allowing the browser to cache files, but still noticing when the files have changed. Toward that end, Propshaft "fingerprints" all the asset files, providing versions of the files with a filename containing a hash value. For example, as I look in the "Sources" tab of my browser tools, I see that the Rails server is sending files named `application-a966e35708de2a027dcba6b1f4b26e1a20cb5c52.css` and `application-a6d77b32a863f62b627bfd9d4c0ce6aca0528329.js`. The extra letters and numbers are the fingerprint. As long as the file's contents stay the same, the browser can cache the fingerprinted filename. When the files change, the fingerprint will change and the browser will know it needs to get a new version from the server.

In development, Propshaft acts as a server, intercepting requests for asset files using Rails helpers like `asset_path` and `image_tag`. In production, the precompile step creates fingerprinted versions of each file along with a manifest that the production server uses to convert the helper requests to the fingerprinted files.

Accessing the Propshaft version of the files is automatic if Propshaft is loaded. The default place that Propshaft looks for files is `app/assets`, though you can change that by setting the `config.assets.paths` attribute at startup. The `image_tag` helper maps to the `/images` directory of the asset path, and the `asset_path` helper maps to the top level asset path.

In CSS files you can use `url` normally to rear to assets using `/assets` to represent the top file, so `url("/assets/images/image.png")` and Propshaft will convert the CSS file to the fingerprinted version automatically.